CSCE3304 Digital Design II – Fall 2026

Finding Cells Worth Adding With AI Used as a Development Aid

GB3:
Kareem Rashed (900233328)
Mahinour Abdelgawad (900231870)
Joshua Makar (900231109)

GitHub Repository URL: https://github.com/Kareem-Rashed/celladvisor
Final Commit Hash :

1. Introduction

When a design repeats a small function that the cell library lacks, the mapper builds it from two or three cells and pays their combined area every time. We ask whether the SkyWater 130 nm high-density library (HD) lacks such cells, and whether adding them makes designs smaller.

Our tool, celladvisor, works in four phases:

a) Find: Synthesize each design, write it as an AND-inverter graph, enumerate every cut of up to 4 inputs, and count how often each function appears.
b) Filter: Drop functions HD already has (Exists, Exists with an inverter, Contained), cost the rest as CMOS cells, estimate their area, and rank them by estimated saving.
c) Measure: Add the candidates to the library, map eight designs with and without them using two ABC scripts, and prove every mapping correct with ABC's CEC.
d) Select: Keep only the cells that lower the total area, and test how much the result depends on the area estimate

When we measure, we compare against the original 189-cell reduced HD library, with the same commands and only the library changed

2. Method

2.1. Flow and baseline

Celladvisor reads each synthesized design as an AIG, enumerates its cuts (up to 4 inputs), classifies each cut's truth table into a normalized form, checks every class against the Sky130 HD library, estimates the areas of the classes HD lacks, and ranks them by estimated saving. Then greedily searches for the final set of candidates and calls cell requests for them, replaces them by real cells and remaps. All of this is simply reproducible by one make command.

The baseline of the project is the reduced HD library. We map designs using the same commands in the handout and we use both ABC scripts. Only the library is changed per run, to test for savings.

2.2. AIG extraction

We synthesize each design using Yosys which writes it as an AIG graph. Async2sync and diffunmap make sure that async resets and enables turn into purely combinational logic. Each design is written three ways: raw (straight from aigmap), strash (merging duplicate nands), and abc (Calls abc to logically restructure the graph).

We compared our results of aig extraction with that of the handout (Table 2), where we found identical flip-flop counts for all designs but slightly lower (6.6 to 29%) raw AND counts which is a result of the different Yosys version we used.

2.3. Cuts

A cut of an AND node n from the AIG is basically a set of at most four leaves, such that each path from an input to n passes through one of the leaves. The logic between the leaves and n

is the function of the cell that we would implement. We start bottom up and join existing cuts as follows: For an AND node *g* with fanins *l* and *r*:

$$\text{cuts}(g) = \{ \{g\} \} \cup \{ x \cup y : x \in \text{cuts}(l), y \in \text{cuts}(r), |x \cup y| \leq 4 \}$$

Where either we stop at the trivial {g} cut, or merge any cut of the left and right fanin, and disregard unions with more than four leaves. Also, any cut that has another cut of the same node as a proper subset is considered dominated, and we remove it.

2.4. Classes

A problem that occurs when checking candidate cells with the HD library is that the functions are not normalized. Take for instance the functions A & B, A &C, and C&D. Clearly, all these functions are just one AND cell where the pins are wired differently, however, they all have different truth tables. Hence, we only compare a cels class with a candidate class, where class id is the canonical form; the smallest integer among the table's variants.

2.5. Redundancy

After extracting all the candidate truth tables, redundancy filtration takes place. Each candidate is compared against every single-output cell in the full HD library and is either kept or dropped according to four categories.

2.5.1. Exists

A candidate function is labeled as *exists* if an HD cell computes the exact same function, and is dropped.

2.5.2. Exists With Inverter

A candidate function is labeled as *exists with inverter* if an HD cell computes the function by adding or removing an inverter (on the output or on a single input) and is dropped, unless the custom single-stage cell is smaller than the existing cell plus an extra inverter.

2.5.3. Contained

A candidate is "Contained**"** if an existing HD cell can implement its function by connecting its pins to inputs A–D or constants 0 and 1, allowing shared inputs.
The checker precomputes all possible pin wirings for each combinational HD cell (up to (6^k) combinations for a (k)-input cell) and stores their resulting 16-bit truth tables in a dictionary. This allows each candidate to be checked with a single lookup.
For each truth table, the cheapest matching HD cell is stored. Following Table 3, a Contained candidate is rejected unless its estimated area is smaller than that of the existing cell. This check is performed after the "Exists" and "Exists with inverter" tests. The table (results/contained_table.csv) holds every reachable 4-input truth table with its cheapest cell and wiring.

2.6. Cost

After the redundancy check, the cost of every candidate function is calculated, whether or not the function is new since the *exists with an inverter* and *contained* rules need the candidate's area to compare against the HD cell.

For functions that can be built as a single CMOS stage, each input used in the formula costs 2 transistors.

No more than 3 transistors can be stacked in series.

If a function cannot be built in one stage, we estimate it as several stages. We count 2 transistors for each input inverter, and 2 more for an output inverter if needed. XOR-like functions cannot be handled this way, so they get a fixed fallback estimate of 40 transistors.

This transistor count is then passed into the area model.

2.7. Area model

We had to estimate the cells we do not have in a quick efficient way so that we can compare it to the HD library. We used the HD library data itself to graph the relationship between transistor count and sites. A site is a fixed height (2.72 um) and a fixed width (0.46um), so a cell size is measured in sites, where height is constant and width is an integer multiple of the fixed 0.46um. We fit the line on the x1 cells only and took out any anomalies that would rather skew the data. We also had a separate fitted line for multi stage cells.

The area model took as input the transistor count of the cell we want to predict, that transistor count came from cost.py (Sec 2.6).

2.8. Ranking

The ranking simply reveals the estimated saving of area, had we replaced every occurrence of this function with a newer cell. The candidates are the 379 kept classes of the ABC graph.

To do that, we write a small verilog module that computes the function from its truth table and map it with the reduced library and both scripts, to get the current area the HD library pays. Then, we use the area model estimate to calculate the saving, and multiply it by the number of occurrences per design.

2.9. Liberty and requests

The selected candidates are added to the existing HD library using liberty_writer.py. For each candidate, the tool generates a Liberty cell definition containing its name, Boolean function, estimated area, input and output pins, and timing information. These new cells are inserted into hd_reduced.lib to create hd_extended.lib, while keeping the original library settings and cells unchanged.

Since the new cells have not yet been physically implemented or characterized, we use placeholder values for input capacitance (0.0024 pF) and rise/fall delays (0.10/0.08 ns). These

values are needed for ABC technology mapping. However, the timing sense of each input is calculated from its truth table to determine whether an input transition can raise the output, lower it, or do either depending on the other inputs.

The candidates in ranked.csv are represented by hexadecimal truth tables. The tool converts these into simplified Sum-of-Products (SOP) Boolean expressions using a Quine-McCluskey. It also generates a JSON request for each new cell, containing its name, inputs, output, Boolean expression, truth table, and estimated area. These requests served as specifications for future physical implementation, while the Liberty file is used for Yosys. Finally, the extended library is validated using two Yosys checks:

a) read_liberty -lib verifies that Yosys can import the library and recognizes all 194 cells (189 original + 5 new).
b) read_liberty performs a more detailed check by parsing the cells' Boolean functions, detecting errors such as references to undeclared input pins.

Together, these checks ensure that the extended library can be imported by Yosys and that its Boolean function definitions can be interpreted correctly before technology mapping.

2.10. Table 5 and CEC

Each of the eight designs is synthesized four times: using the original and extended libraries with ABC scripts A and B. We record the area and number of new cells used. Table 5 compares the results for each script, while the best-of-both comparison selects the smaller area from each library to measure the best possible improvement.

The mapped Verilog is saved after measuring the area (stat) to keep the synthesis process consistent with the handout, since saving it earlier was found to affect ABC's mapping results.

Finally, we use ABC's Combinational Equivalence Checking (CEC) to verify that the mapped circuit produces the same logic as the reference circuit before ABC mapping. Both circuits are converted using the Liberty cell functions and compared. Flip-flops are matched by order because their output names may differ.

2.11. Greedy selection, curve

The greedy selection process starts with the baseline library mapping area, and then, for each new candidate, we add that cell to the library and remap, checking the area optimization each time. We greedily do this process until either no candidate helps with the area, or we reach 10 candidates, as per the handout.

This part is crucial to our final results and so we made two critical decisions in the process:

1. We only use Script B in the mapping, since we realized that script A has an odd behaviour where the area changes with the library even when no cell is used. This is shown in the best of both graph in our results. So to make sure we are greedily choosing based on this cell’s effect only, we went with Script B.

2. We only accept the cell mapping if every cell of the library appears in the mapped netlist. The reason is simple, since we do not understand the inner workings of ABC, sometimes the mapping appears to decrease the area of the design while not even using the newer cell at all. Hence, we decided to add a condition that not only shall the area decrease, but also maintain using all cells up to this cell we just added.

After the greedy selection, we plot a cell-by-cell curve. The final cells are added to the baseline library one at a time, in the order the greedy chose them. At each step, every design is mapped with the extended library using both Script A and Script B, and for each design the smaller of the two areas (best of both) is kept. These are added up to give the total area at that step, and the change from the previous step shows how much that cell contributes. This is done with the estimated areas and with the real areas from the cell generator.

2.12. Real cells

Each cell of the final set we got from the greedy selection is written as a request in the format of Appendix A of the handout. We then use the cell generator given to us by Dr. Shalan to use the actual area in the liberty file to then remap the 8 designs. Also, the generator has checks we call on all our <=10 candidates, which sometimes leads to dropping some that fail the checks.

2.13. 30% test

To test how much our results depend on the estimated cell areas, we reduce the area of the first cell selected by the greedy algorithm by 30% in the Liberty file, keeping everything else unchanged. We then map all eight designs twice using ABC Script B: once with the original estimated areas and once with the reduced area. For each design, we compare the total mapped area and how many times the modified cell is used. The results are saved in sensitivity.csv, showing whether making the cell cheaper leads to better area savings or causes ABC to use it more frequently.

3. Verification

Every check is shown to fail on a deliberately broken case, and all of them run in *make test*.

Check	**Passes on**	**Broken case (must fail)**
Cut truth table	0x0707 for a cut of a small hand-made AIG	The same cut claimed as 0x0706 (one bit flipped)
Redundancy checker	Every HD cell of up to 4 inputs is Exists; 0x077F is New	nand2 (0x7777) given as a candidate must not be New
Contained table	nand3 with a pin tied to 1 equals nand2; the cheapest drive strength is kept	XOR (A ⊕ B) looked up in a table built from only nand3 and mux2 must not be found as Contained
Liberty writer	The handout's example cell is reproduced	Adding a cell whose name is already in

Check	Passes on	Broken case (must fail)
	exactly; its request is 0x077F, as in Appendix A	the library must be refused
Liberty file loads	Yosys imports 189 + N cells and parses every function	A file with a missing brace; a function that names a pin the cell lacks
Equivalence (*cec*)	cpu8 mapped with script A	The same netlist with one nand2 replaced by nor2
30% experiment	Only the shrunk cell's area changes, by exactly 30%; the other cells and their order are unchanged. Without the shrink, the final cells give the greedy's script B total exactly (93,662.33 µm²)	Shrinking a cell that is not in the final set must be refused

Table 1. Verification checks

With these checks, the redundancy checker reproduces the handout's Table 4 exactly: 24 Exists, 20 Exists with an inverter, 38 Contained and 3,822 New, over the 3,904 classes of 4-input functions that use all four inputs.

4. Results

All 24 runs of the handout's Table 1 work, and our areas are within about 4.5% of the handout's, mostly smaller, because we use Yosys 0.70 instead of 0.33. The reduced library costs script B almost nothing but costs script A up to 3.0% (fir10), where the handout reports at most 0.5%; the full table is in results/table1.csv.

Design	AND nodes	Cuts	P classes	PN classes	NPN classes
cpu8	884 → 855	6,611 → 6,032	211 → 202	172 → 169	57 → 53
dsp8	1,361 → 1,303	11,830 → 10,758	268 → 221	206 → 180	54 → 49
arlet6502	2,426 → 2,102	20,106 → 16,710	298 → 261	198 → 185	55 → 47
uart_apb	2,215 → 1,746	16,248 → 10,372	199 → 140	141 → 107	44 → 30
risc8	5,211 → 4,865	42,859 → 36,109	279 → 232	190 → 168	52 → 42
fir10	6,492 → 6,385	70,081 → 66,698	258 → 240	207 → 200	55 → 55
spm	380 → 368	2,856 → 1,933	46 → 35	44 → 35	22 → 14
mul	525 → 510	6,044 → 5,782	200 → 185	157 → 146	41 → 34

Table 2. Classes and Cuts before and after ABC restructuring

Step	Cell	Instances	Change (µm²)	Estimated (µm²)	Real (µm²)	HD cost (µm²)	Simulation
1	c033f	384	-904.6	8.76	20.02	12.51	pass
2	c000b	188	-203.9	8.76	23.77	11.26	fail
3	c2abf	5	-75.1	12.51	137.63	17.52	fail
4	c0aff	48	-37.5	7.51	21.27	8.76	pass
5	c0aaf	106	-18.8	11.26	23.77	12.51	pass
6	caaee	7	-15.0	8.76	27.53	11.26	pass
7	c037f	2	-8.8	10.01	36.28	13.76	fail
8	c03ab	15	-15.0	8.76	25.02	10.01	pass
9	c002a	5	-10.0	8.76	23.77	11.26	fail
10	c02ff	5	-12.5	8.76	18.77	11.26	pass

Table 3. Greedy selection (script B total 94,963.58 → 93,662.33 µm²): change when each cell is added, estimated and real area, HD cost and the generator's simulation check. All cells pass the layout checks; c2abf has an isolated layout (110 sites).

Areas used	Script B, step 0 to 10	Best of both, step 0 to 10	Best of both, lowest and highest point	Instances
Estimated	94,963.58 → 93,662.33 (-1.37%)	91,273.79 → 91,337.60 (0.07% larger)	90,400.45 and 91,428.94	717
Real	94,963.58 → 95,012.37 (+0.05%)	91,273.79 → 91,258.77 (0.02% smaller)	89,709.79 and 91,273.79 (-1.71%)	0

Table 4. Best-of-both area (µm²)

Design	Baseline	Estimated cells	Reduction	Real cells	Reduction	Baseline delay (ns)	Estimated delay (ns)	Real delay (ns)
cpu8	4,195.3	4,194.0	+0.03%	4,187.8	+0.18%	4.11	4.15	4.09
dsp8	5,794.3	5,833.1	-0.67%	5,791.8	+0.04%	4.97	5.14	4.97
arlet6502	9,320.2	9,334.0	-0.15%	9,316.4	+0.04%	6.94	6.52	7.02

Design	Baseline	Estimated cells	Reduction	Real cells	Reduction	Baseline delay (ns)	Estimated delay (ns)	Real delay (ns)
uart_apb	12,817.3	12,827.3	-0.08%	12,817.3	+0.00%	3.93	6.10	3.93
risc8	28,109.5	28,122.0	-0.04%	28,108.2	+0.00%	24.77	24.88	24.77
fir10	26,322.7	26,326.5	-0.01%	26,322.7	+0.00%	7.08	7.05	7.08
spm	2,869.0	2,869.0	+0.00%	2,869.0	+0.00%	0.81	0.73	0.81
mul	1,845.5	1,831.8	+0.75%	1,845.5	+0.00%	4.59	4.44	4.59
Total	**91,273.8**	**91,337.6**	**-0.07%**	**91,258.8**	**+0.02%**	-	-	-

Table 5. OpenSTA worst delay (ns) per design: baseline, with estimated cells and with real cells.

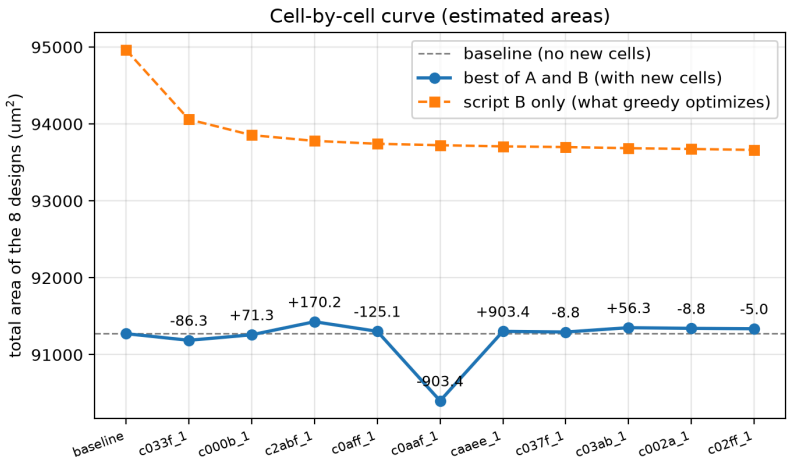


Figure 1. Cell-by-cell curve

5. Discussion

5.1. Classes: Why PN Matters More Than NPN

Across the eight designs, the ABC graphs contain 306 classes up to input order and output inversion (PN), but only 79 NPN classes, about four times fewer. Of the PN classes, 274 use all four inputs, and these fall into 69 NPN classes. NPN treats inverters on the inputs as free. In CMOS each inverter costs 2 transistors, so NPN puts cheap and expensive functions into one class. For example, nand2, nor2, and2 and or2 are one NPN class. But nand2 and nor2 take 4 transistors in one stage, and and2 and or2 take 6 transistors in two stages. PN keeps these apart, so the PN count is the one that matters for choosing cells. ABC restructuring lowers the class count in almost every design (for example uart_apb goes from 141 to 107 PN classes).

5.2. Redundancy

On the ABC graphs there are 450 classes in total. 44 already exist in HD and are dropped. 27 exist with an inverter, and we kept 25 of them. 33 are contained in an HD cell, and we kept 8. The other 346 are new and kept. That leaves 379 candidates and 71 dropped classes. The checker reproduces the handout's Table 4 exactly. For example, class 000b is a211oi_1 plus an inverter (11.26 µm²), and it was kept because its estimated area (8.76 µm²) is smaller. This matches the handout's expectation of small gains, because HD already covers most small single-stage functions. Note that the 33 kept candidates from the inverter and contained groups are only kept because of the area estimate (see 5.5).

5.3. Which Cells Did the Work

The greedy selection stopped at 10 cells and lowered the script B total from 94,963.58 to 93,662.33 µm² (−1.37%). Most of this comes from the first cells. c033f alone saves 904.6 µm², about 70% of the total, and the first three cells give about 91%. c033f is the complement of the majority of three inputs, which is the kind of carry an adder computes. The later cells each save between 8 and 38 µm².

5.4. Best of Both, and Why the Saving Disappears

The gain from script B does not survive the best-of-both comparison. Without any new cell, the best-of-both baseline (91,273.79 µm²) is already about 3.9% smaller than the script B baseline, and the script B total with all 10 new cells (93,662.33 µm²) is still 2.6% above it. With estimated areas the final best-of-both total is 91,337.60 µm², which is 0.07% larger than the baseline. With real areas it is 91,258.77 µm² (0.02% smaller), but the real cells are used 0 times, so that change is not caused by the cells. Only cpu8 and mul improve with estimated cells, and the other designs get slightly larger.

The cell-by-cell curve is not smooth. With estimated areas the total drops by 903 µm² when c0aaf is added, then rises by exactly 903 µm² when caaee is added, and ends at the same total as step 4 (91,303.82 µm²). A cell that is added and then cancelled cannot have caused the drop. Most of this swing comes from fir10 under script A (±1,305 µm²). The same effect appears in Table 1: removing 118 cells the design does not use changes fir10 by 3.0%. So script A reacts to the contents of the library even when new cells are not used, and this is why the greedy selection used script B only. It also means the lowest point on the curve (90,400.45 µm², −0.96%) is not a real saving.

The new cells are used unevenly across designs. With the 10-cell library, c033f appears 288 times in fir10, 19 in dsp8 and 9 in mul, but 0 times in uart_apb and spm, so the benefit goes to the datapath-heavy designs. We mapped only risc8 from the risc8/pic16/p16c5x family, so we cannot say whether the three designs favour the same cells.

5.5. Frequency Against Saving

A frequent function does not always save area. The saving per occurrence is what HD pays now minus the cost of the new cell. Class 000e is the second most frequent (2,375

occurrences), but HD builds it for 8.76 µm² and our new cell would cost 11.26 µm², so it loses 5,943 µm² in total. The XOR-like class 9996 has 1,927 occurrences, but the new cell would cost 38.79 µm² against 21.27 µm² in HD, so it loses 33,755 µm². Of the 379 ranked classes, only 69 are estimated to save area and 298 lose it. The same happens in the greedy mapping: c0aaf is used 106 times but saves only 18.8 µm², while c2abf is used 5 times and saves 75.1 µm².

5.6 Estimated vs. Real Areas, Timing and Sensitivity

The real cells are much larger than estimated. Nine of the ten are 2.1 to 3.6 times larger than their estimates, and c2abf is 11 times larger (137.63 µm², isolated layout). Every real cell is larger than what HD already pays for the same function (for example c033f is 20.02 µm² against 12.51 µm²). So with real areas, all ten candidates would have been dropped by the redundancy rule. A likely reason is that the area model was fitted on HD's hand-optimized cells, while the generator's layouts are less compact. Four real cells also fail the generator's simulation check (c000b, c2abf, c037f, c002a).

For timing, the estimated cells use placeholder delays, so the OpenSTA numbers only show what the mapper did. The worst case is uart_apb, which goes from 3.93 to 6.10 ns. Arlet6502 gets faster (6.94 to 6.52 ns). With real cells the delays are almost unchanged.

The 30% test shows how strongly the result depends on the estimate. Making c033f 30% smaller lowers the script B total by 1,072.5 µm² (about 1.15%) and raises its use from 318 to 472 instances, mostly in fir10 (−947.7 µm²). One cell's area changes the total about as much as all ten cells save in the first place. Since the real cells are larger than estimated, the real error goes in the wrong direction.

In summary, with HD as the baseline we found no reliable saving from the new cells. The small savings the handout predicted exist only with estimated areas and script B, and they disappear with real areas and the best of both scripts.

6. Limitations

1. Yosys 0.70 instead of the handout's 0.33. Our AND counts are 6.6% to 29.3% lower, Table 1 areas differ by -4.49% to +2.22%, and the reduced library costs script A up to 3.00% (handout: under 0.5%).
2. Script A is not stable: its area changes with the library even when no new cell is used, and it differs between machines; the cause is not known. Best of both moves by up to 1,564 µm² (1.7%) with unused cells (see 5.4), and all numbers come from one machine and one run.
3. Script B has a noise floor of about 250 µm² when a cell is added and not used. Only c033f clearly exceeds it; the last eight selected cells are not evidence of savings.
4. The selection scores script B, and the total covers eight designs. The handout does not specify the script; the final set is evaluated on both scripts and on best of both.
5. Estimated areas are optimistic: the model describes HD's hand-optimized layouts. Real cells are 2.1 to 3.6 times the estimate and 1.6 to 2.6 times HD's cost (c2abf: 11 and 8 times); see 5.6.
6. The generator: 14 of the first 30 ranked candidates could not be laid out, and 4 of the 10

final cells fail its logic simulation. Only 6 cells are valid real cells; the real library holds all ten, and none is used.

7. Contributions

Joshua Makar: the Contained test; the broken-case tests for the cut and redundancy checkers; the Liberty writer, the cell requests and the Liberty check; Table 5 mapping with equivalence checking and its broken cases; the module interface document.

Mahinour Abdelgawad: Cost estimator; cell-by-cell curve; cuts truth table extractor and checker; HD library parser; redundancy checker; NPN class counts; estimate_area() function in areamodel.

Kareem Rashed: AIGs of every design, AIGER reader and the cut enumerator; the cut counts per design; the canonical form of the truth tables,the area model; the ranking; the greedy selection; the real areas from the cell generator, its checks; the OpenSTA delays.

Appendix: Tools Used

- **Yosys:** 0.70+ with bundled yosys-abc
- **Python:** 3.15
- **OpenSTA:** 3.1.0